



**SIMATS**  
ENGINEERING



**SIMATS**  
Saveetha Institute of Medical And Technical Sciences  
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

# **Design of a QR-Based Event Registration & Pass System Using Python**

## **A CAPSTONE PROJECT REPORT**

*A Capstone Project Report submitted in partial fulfilment of the requirements  
for the Course of*

**CSA0801 – PYTHON PROGRAMMING**

*to the award of the degree of*

**BACHELOR OF TECHNOLOGY**

**IN**

**COMPUTER SCIENCE AND ENGINEERING**

**Submitted by**

**Harini K (1973260001)**

**Nithyasree P (1911260096)**

**S R Anusika (1920260008)**

**Keerthi P (1921260078)**

**Under the Supervision of / Course Faculty**

**Dr. U. Samson Ebenezar**

**SIMATS ENGINEERING**

**Saveetha Institute of Medical and Technical Sciences**

**Chennai-602105**

**SEPTEMBER 2026**

# TABLE OF CONTENTS

| Sl. No | Title                                                                                       | Page No |
|--------|---------------------------------------------------------------------------------------------|---------|
| i      | CERTIFICATE FROM THE SUPERVISOR                                                             | ii      |
| ii     | DECLARATION BY THE CANDIDATE                                                                | iii     |
| iii    | INDIVIDUAL CONTRIBUTION STATEMENT                                                           | iv      |
| iv     | ABSTRACT                                                                                    | v       |
| v      | LIST OF FIGURES                                                                             | vi      |
| vi     | LIST OF TABLES                                                                              | vii     |
| vii    | LIST OF ABBREVIATIONS                                                                       | viii    |
| 1      | CHAPTER 1: Introduction and Engineering Problem                                             | 1       |
| 2      | CHAPTER 2: Literature Review and Existing Solutions                                         | 5       |
| 3      | CHAPTER 3: Engineering Design & Trade-offs, Sustainability, Ethics, Safety, Risk & Security | 8       |
| 4      | CHAPTER 4: Implementation, Testing & Results, Project Management                            | 12      |
| 5      | CHAPTER 5: Conclusion, Future Work and Reflection                                           | 16      |
|        | References                                                                                  | 18      |
|        | Appendices                                                                                  | 19      |

# **SIMATS ENGINEERING**

**Saveetha Institute of Medical and Technical Sciences**

**Chennai-602105**

## **BONAFIDE CERTIFICATE**

This is to certify that the Capstone Project entitled “Design of a QR-Based Event Registration & Pass System Using Python” has been carried out by Harini K (Reg. No. 1973260001), Nithyasree P (Reg. No. 1911260096), S R Anusika (Reg. No. 1920260008) and Keerthi P (Reg. No. 1921260078) under the course faculty Dr. U. Samson Ebenezer and is submitted in partial fulfilment of the requirements for the current semester of the B.Tech Computer Science and Engineering program at Saveetha Institute of Medical and Technical Sciences, Chennai.

### **COURSE COORDINATOR**

**Dr. S. Loganayagi**

**Professor**

**Department of CSE**

**SIMATS Engineering,**

**SIMATS, Chennai – 602105.**

### **COURSE FACULTY**

**Dr. U. Samson Ebenezer**

**Professor**

**Department of CSE**

**SIMATS Engineering,**

**SIMATS, Chennai – 602105.**

# **SIMATS ENGINEERING**

**Saveetha Institute of Medical and Technical Sciences**

**Chennai-602105**

## **DECLARATION**

We, Harini K, Nithyasree P, S R Anusika and Keerthi P of the Department of Computer Science and Engineering, SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai, hereby declare that the Capstone Project Work entitled “Design of a QR-Based Event Registration & Pass System Using Python” is the result of our bonafide efforts. To the best of our knowledge, the work presented herein is original and has been carried out with academic integrity. All sources, software tools, datasets and other materials used in the project have been appropriately acknowledged. We confirm that the implementation, test outputs and findings presented in this report are based on the project work described in the submitted presentation.

Place: Chennai

Date: September 2026

Signatures of the Students:

Harini K

Nithyasree P

S R Anusika

Keerthi P

# SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

## Individual Contribution Statement

| Student Name / Register No. | Specific Responsibilities                   | Design & Development Contribution                  | Testing & Analysis Contribution                    | Report Contribution                   | Approx. Contribution (%) |
|-----------------------------|---------------------------------------------|----------------------------------------------------|----------------------------------------------------|---------------------------------------|--------------------------|
| Harini K / 1973260001       | Team registration module; UI input handling | Registration workflow and participant data capture | Registration validation and duplicate-email checks | Report sections and presentation      | 25%                      |
| Nithyasree P / 1911260096   | QR generation and pass handling             | Unique ticket ID and QR/Base64 generation          | QR round-trip verification                         | Implementation and results sections   | 25%                      |
| S R ANUSIKA / 1920260008    | Gate verification and attendance logic      | Ticket validation, check-in and duplicate blocking | Verification and attendance-status testing         | Report and presentation documentation | 25%                      |
| Keerthi P / 1921260078      | Admin roster and telemetry                  | Statistics, roster display and session control     | Telemetry and access-control checks                | Report and presentation documentation | 25%                      |
| Total                       |                                             |                                                    |                                                    |                                       | 100%                     |

# ABSTRACT

This capstone project presents the design of a lightweight QR-based event registration and pass system using Python. The engineering problem addressed is the slow, insecure and error-prone nature of manual participant registration and gate attendance tracking at large events such as hackathons. Paper sheets and spreadsheets can create gate congestion, duplicate check-ins, proxy entries and difficulty in maintaining a live attendance record. The proposed system provides an interactive registration interface in which participants enter team name, leader details, email, team size and domain track. A unique ticket identifier is generated for each registration, and a QR pass is created dynamically in memory using a QR-code library, BytesIO and Base64 encoding. The system stores registration and check-in records locally using SQLAlchemy with an SQLite database, avoiding dependence on a third-party database service. Gate verification is protected by a master passcode before the camera scanner is activated. On scanning, the ticket ID is checked against the database; valid tickets are accepted, invalid tickets are rejected and duplicate check-in attempts are blocked. An administrator telemetry module provides registered-team, checked-in-team, total-participant and participant-inside statistics together with a live roster. The development follows a Waterfall approach covering requirement analysis, interface design, QR generation, database handling, gate verification and testing. The presentation reports successful registration, QR generation, verification and attendance tracking across multiple test runs without data loss. It also reports 100% matching ticket payloads across round-trip QR test cases and correct blocking of unauthorized access attempts. The project demonstrates how Python programming, web application development, database modeling and QR-based authentication can be combined to solve a practical event-management problem. The main contribution is a compact system architecture that reduces manual gate work while maintaining controlled access and persistent attendance records.

**Keywords:** Python; QR code; event registration; SQLite; attendance verification; FastHTML

## LIST OF FIGURES

| Figure No. | Figure Name                                                      | Page No. |
|------------|------------------------------------------------------------------|----------|
| 1          | System flow of QR-based event registration and gate verification | 9        |
| 2          | Admin attendance telemetry workflow                              | 14       |

## LIST OF TABLES

| Table No. | Table Name                                 | Page No. |
|-----------|--------------------------------------------|----------|
| 1         | Literature review comparison               | 7        |
| 2         | Stakeholder requirements                   | 9        |
| 3         | Functional and non-functional requirements | 10       |
| 4         | Test plan and verification                 | 13       |
| 5         | Results and objective achievement          | 15       |

## LIST OF ABBREVIATIONS / SYMBOLS

| Symbol / Abbreviation | Description                      | Unit |
|-----------------------|----------------------------------|------|
| QR                    | Quick Response code              | —    |
| UI                    | User Interface                   | —    |
| ORM                   | Object-Relational Mapping        | —    |
| HTML5                 | HyperText Markup Language<br>5   | —    |
| SMTP                  | Simple Mail Transfer<br>Protocol | —    |

# CHAPTER 1

## INTRODUCTION AND ENGINEERING PROBLEM

### 1.1 Background

Event registration and gate attendance are important operational tasks in hackathons and other large events. Manual registration through paper sheets or spreadsheets is slow and error-prone, while manual checking can create long queues and make duplicate or proxy entries difficult to control. The proposed project uses Python to build a lightweight web application that combines participant registration, digital QR FastPass generation, controlled gate scanning and live attendance telemetry in one workflow.

### 1.2 Need for the Project

Large events require fast and reliable participant verification at entry points. The PPT identifies gate congestion, long queues, proxy entries and untracked duplicate check-ins as practical problems. A digital ticket identifier and QR pass can make verification more structured, while a database can maintain persistent registration and check-in records. Administrative access control is also required so that attendee telemetry and optical camera verification are not openly accessible.

### 1.3 Problem Statement

How can a lightweight Python-based event registration system reduce manual gate effort while generating unique QR passes, securely controlling verification functions, persisting registration records and preventing duplicate check-ins during live event entry?

### 1.4 Project Aim

To design and implement a Python-based QR event registration and pass system that supports participant registration, unique digital passes, controlled gate verification and live attendance tracking.

### 1.5 Project Objectives

- Allow participants to save team name, member/leader details, email, squad size from 1 to 5 and domain track.
- Generate dynamic QR passes containing unique ticket IDs without requiring external file hosting.
- Persist registration and check-in records locally using SQLAlchemy and SQLite.
- Protect camera verification and attendance telemetry using a master passcode.
- Validate scanned ticket IDs and block duplicate check-in attempts.

Display live roster and attendance statistics for authorized organizers.

### 1.6 Scope

The project includes participant registration, team-size validation, duplicate leader-email checking, unique ticket-ID generation, in-memory QR creation, Base64 image encoding, SQLite persistence, passcode-controlled gate verification, QR ticket lookup, duplicate check-in

prevention and administrator attendance telemetry. The project is focused on the lightweight Python web application described in the PPT. It does not include enterprise identity management, distributed multi-gate deployment, automated email delivery, PostgreSQL migration or NFC/RFID integration; these are identified as future enhancements.

### **1.7 Expected Engineering Outcome**

The expected outcome is a functional lightweight event-management application in which a participant registration produces a unique digital QR pass, an authorized gatekeeper can verify the ticket, duplicate entries are blocked, and administrators can view live attendance statistics and roster information.

### **1.8 Project Significance**

The project demonstrates practical application of Python programming to a real-world operational problem. It combines web input handling, QR generation, database persistence, session-based access control, optical verification and live data aggregation in a single workflow.

## CHAPTER 2

### LITERATURE REVIEW AND EXISTING SOLUTIONS

#### 2.1 Review Method

The literature survey in the submitted PPT reviewed five works related to QR-based access control, in-memory data streaming, Python ORM/database modeling, automated ticketing and modern Python web application development. The review compared the stated methodology and findings of each work with the requirements of the proposed event registration and pass system.

#### 2.2 Review of Existing Work

Patel et al. (2021) examined QR-code-based access control using 2D QR encoding and mobile camera decoding. Zhang et al. (2022) discussed in-memory binary buffering with BytesIO and Base64 data URLs. Rao et al. (2023) discussed SQLAlchemy ORM with SQLite for lightweight database management. Sharma and Kumar (2023) considered relational ticketing with unique UUID-based identifiers and duplicate detection. Gupta and Verma (2024) discussed modern Python architectures for responsive web applications. Together, these works support the main technologies selected in the project.

#### 2.3 Comparative Analysis

| Existing Approach                | Advantages                 | Limitations                                   | Relevance to Proposed Work         |
|----------------------------------|----------------------------|-----------------------------------------------|------------------------------------|
| Paper / spreadsheet registration | Simple to start            | Slow; error-prone; weak duplicate control     | Problem being addressed            |
| QR-based access control          | Fast digital verification  | Requires camera and decoding                  | Core verification approach         |
| Cloud/distributed database       | Supports multiple gates    | More infrastructure and deployment complexity | Future PostgreSQL extension        |
| Local SQLite                     | Lightweight and persistent | Requires deployment persistence handling      | Selected for current project       |
| Manual attendance checking       | Low setup cost             | Queues and untracked duplicate entries        | Replaced by automated verification |

#### 2.4 Research / Engineering Gap

The reviewed approaches address individual parts of access control, data handling, database modeling and web development. The project combines these functions into one lightweight workflow specifically aimed at event registration and live gate management. The engineering gap addressed by the project is therefore integration: registration, QR generation, controlled scanning, persistent attendance and live telemetry must operate together with minimal infrastructure dependency.

#### 2.5 Proposed Contribution

The proposed contribution is an integrated Python application that combines interactive team registration, unique PDX26-XXXXXX ticket identifiers, in-memory QR generation, SQLite persistence, passcode-controlled optical verification, duplicate check-in prevention and an administrator telemetry view. This integration provides a practical foundation for event entry management while remaining lightweight.

## CHAPTER 3

### ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY

#### 3.1 Requirements

| Stakeholder / User         | Requirement                                                          |
|----------------------------|----------------------------------------------------------------------|
| Participants               | Enter valid team and leader details and use the generated QR pass    |
| Gatekeeper / Organizer     | Verify tickets only after authorization and handle entry decisions   |
| Administrator              | Monitor live roster and attendance telemetry                         |
| Developers / Engineers     | Validate input, design reliable modules and test system behaviour    |
| Event Management           | Maintain accurate registration records and efficient gate operations |
| Hosting / Deployment Team  | Ensure persistence, HTTPS and camera permissions where required      |
| Institution / Faculty      | Evaluate the capstone implementation and documentation               |
| Future Multi-Gate Operator | Requires distributed storage and multi-user authentication           |

#### Functional & Non-Functional Requirements

| Requirement Type | Measurable/Assessable Target                                   |
|------------------|----------------------------------------------------------------|
| Functional       | Register team name, leader, email, squad size (1–5) and domain |
| Functional       | Generate a unique PDX26-XXXXXX ticket ID and QR pass           |
| Functional       | Store registration and check-in data in SQLite                 |
| Functional       | Validate tickets and block duplicate check-ins                 |
| Non-Functional   | Require authorization before gate scanning and telemetry       |
| Non-Functional   | Provide lightweight, maintainable local deployment             |

#### 3.2 Constraints & Applicable Standards

The PPT does not specify a named external engineering standard. The main constraints explicitly identified are lightweight deployment, local SQLite persistence, browser camera permissions and HTTPS requirements for camera scanning, a shared organizer master passcode, and the

limitations of free cloud instances such as cold-start delays. These constraints are treated as project design conditions rather than replaced with unrelated standards.

### 3.3 Design Specifications

| Parameter          | Target / Requirement                               | Unit      | Priority |
|--------------------|----------------------------------------------------|-----------|----------|
| Ticket ID          | Unique identifier in PDX26-XXXXXX pattern          | ID        | High     |
| Team size          | Accept only 1–5 members                            | members   | High     |
| QR payload         | Must match stored ticket ID after decoding         | ID        | High     |
| Attendance status  | First valid scan sets attended=True with timestamp | condition | High     |
| Duplicate check-in | Second scan must be blocked with prior timestamp   | condition | High     |
| Admin access       | Master passcode required for protected modules     | condition | High     |

### 3.4 Alternative Solutions & Evaluation

Alternative 1: Continue with paper or spreadsheet-based registration and manual gate checking. This has low software complexity but does not provide dynamic QR passes, automated duplicate detection or live telemetry.

Alternative 2: Use the proposed Python web application with QR generation, SQLite persistence, passcode-controlled scanning and telemetry. This provides an integrated workflow while keeping infrastructure requirements relatively low.

Alternative 3: Extend the system directly to distributed cloud storage and multi-gate hardware. This can support larger deployments but introduces greater infrastructure and implementation complexity than the current project requires.

| Criterion               | Weight | Alternative 1 | Alternative 2 | Alternative 3 |
|-------------------------|--------|---------------|---------------|---------------|
| Integration performance | High   | Low           | High          | High          |
| Implementation cost     | Medium | Low           | Medium        | High          |
| Access control          | High   | Low           | High          | High          |
| Scalability             | Medium | Low           | Medium        | High          |
| Maintenance             | High   | Medium        | High          | Medium        |

### 3.5 Selected Design & Detailed Design

Alternative 2 is selected because it directly addresses the problems stated in the PPT while remaining lightweight. It provides digital registration, unique ticketing, database persistence, controlled verification and live attendance information without requiring the future infrastructure additions listed in the presentation.

**Conceptual System Flow: Participant Registration → Duplicate Email Check → Unique Ticket ID → QR Generation/Base64 Encoding → SQLite Storage → Admin Passcode → Camera Scan → Ticket Validation → First Check-in / Duplicate Warning → Live Telemetry.**

Systems approach: The registration module creates the participant record and ticket ID, while the QR-generation function converts the ticket ID into a browser-ready image. The gate-verification module reads the scanned ticket, validates it against the database and updates attendance status. The telemetry module aggregates the same database records to display registered teams, checked-in teams and participant counts, so the three modules depend on a common persistent data model.

### 3.6 Trade-off Analysis

| Design Decision     | Alternative Considered | Benefit              | Disadvantage                      | Final Decision & Justification                      |
|---------------------|------------------------|----------------------|-----------------------------------|-----------------------------------------------------|
| Registration method | Paper/spreadsheet      | Low setup complexity | Manual errors and slow processing | Use web input with validation                       |
| Pass generation     | Static/manual pass     | Simple               | No unique dynamic verification    | Generate unique QR in memory                        |
| Database            | External cloud DB      | Scalable             | Higher dependency                 | Use SQLite for current lightweight scope            |
| Authentication      | Open admin tools       | Easy access          | No protection                     | Use master passcode; improve to role accounts later |

### 3.7 Sustainability, Ethics, Safety, Risk & Security

| Domain          | Consideration / Evidence Evaluated                                                              | Impact on Final Decision                                                                              |
|-----------------|-------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|
| Sustainability  | Lightweight architecture and local SQLite reduce infrastructure dependency in the current scope | Retain lightweight implementation while identifying PostgreSQL only for future distributed deployment |
| Ethics          | Participant records and access decisions should be handled responsibly                          | Restrict telemetry and scanning behind organizer authentication                                       |
| Health & Safety | Fast, controlled gate                                                                           | Use digital verification and                                                                          |

|          |                                                                                 |                                                    |
|----------|---------------------------------------------------------------------------------|----------------------------------------------------|
|          | processing can reduce congestion and entry confusion                            | duplicate blocking                                 |
| Security | Unauthorized access to scanning and telemetry could compromise event operations | Require master passcode before protected functions |

## Risk Register

| Risk                     | Likelihood | Severity | Risk Level | Mitigation                                               | Residual Risk |
|--------------------------|------------|----------|------------|----------------------------------------------------------|---------------|
| Duplicate registration   | Possible   | Medium   | Medium     | Check duplicate leader email                             | Reduced       |
| Invalid ticket scanned   | Possible   | High     | High       | Validate ticket ID in database                           | Reduced       |
| Duplicate check-in       | Possible   | High     | High       | Attendance flag + timestamp check                        | Reduced       |
| Unauthorized gate access | Possible   | High     | High       | Master passcode before scanner                           | Reduced       |
| Camera unavailable       | Possible   | Medium   | Medium     | HTTPS + browser permissions; manual operational fallback | Reduced       |

# CHAPTER 4

## IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT

### 4.1 Development Approach & Resources

The project follows a Waterfall development approach consisting of requirement analysis, UI design, QR generation logic, database handling, gatekeeper verification and testing. The main implementation uses Python with FastHTML for the web interface, SQLAlchemy for database ORM, SQLite for local persistence, and a QR-code library with BytesIO and Base64 encoding for dynamic pass creation. Client-side HTML5 camera stream integration is used for optical scanning.

### 4.2 Software / Research Resources & Modern Tools Used

| Tool / Resource                    | Purpose                          | Stage Used     |
|------------------------------------|----------------------------------|----------------|
| Python                             | Core implementation language     | Throughout     |
| FastHTML                           | Web interface and input workflow | Implementation |
| SQLAlchemy                         | Database ORM and record handling | Implementation |
| SQLite                             | Local persistent database        | Implementation |
| QR-code library + BytesIO + Base64 | Dynamic QR image generation      | Implementation |

### 4.3 Test Plan & Verification

Verification focused on the functional requirements stated in the PPT: required-field validation, unique ticket creation, QR payload matching, database persistence, passcode protection, valid-ticket acceptance, invalid-ticket rejection, duplicate check-in blocking and telemetry aggregation. The test plan compares each requirement with the expected system behaviour rather than introducing measurements not reported in the presentation.

| Requirement                  | Test / Verification Method                   | Acceptance Criterion                             |
|------------------------------|----------------------------------------------|--------------------------------------------------|
| Required registration fields | Submit incomplete and complete registration  | Incomplete input rejected; complete input stored |
| QR generation                | Generate and decode ticket QR                | Decoded payload matches ticket ID                |
| Ticket verification          | Scan valid and invalid IDs                   | Valid accepted; invalid rejected                 |
| Duplicate check-in           | Scan same valid ticket twice                 | Second attempt blocked with previous timestamp   |
| Admin protection             | Attempt protected functions without passcode | Unauthorized access blocked                      |

## 4.4 Results

The PPT reports successful saving, generation, verification and tracking of participant teams across multiple test runs without data loss. The master passcode correctly blocked unauthorized attempts to access gate verification tools. QR generation and optical decoding produced 100% matching ticket payloads across the reported round-trip test cases. A demonstration telemetry output showed 45 teams registered, 32 teams checked in, 180 total hackers and 128 hackers inside the venue.

| Finding                     | Cases / Evidence in PPT                                        | Engineering Implication                             |
|-----------------------------|----------------------------------------------------------------|-----------------------------------------------------|
| QR payload matching         | 100% matching ticket payloads across reported round-trip tests | Supports correctness of QR generation/decoding flow |
| Unauthorized access control | Every reported unauthorized attempt was blocked                | Supports protected scanner/telemetry design         |
| Registration and tracking   | Successful across multiple test runs without data loss         | Supports SQLite-backed workflow                     |
| Telemetry demonstration     | 45 registered teams; 32 checked in; 180 hackers; 128 inside    | Shows live aggregation capability                   |
| Duplicate detection         | Second scan returned duplicate warning with prior time         | Prevents repeated ingress                           |

## 4.5 Data Analysis & Engineering Judgment

The reported results indicate that the core data flow is consistent: a ticket ID generated during registration can be encoded into a QR pass, decoded at the gate and matched with the stored database record. Duplicate detection is based on the attendance flag and timestamp. The 100% payload matching result supports the correctness of the QR round-trip tests reported in the PPT. The remaining limitations are operational rather than failures of the basic data model, particularly camera permissions, HTTPS requirements, shared authentication and deployment persistence.

## 4.6 Comparison with Existing Solutions & Achievement of Objectives

| Objective                  | Evidence                                     | Achievement    |
|----------------------------|----------------------------------------------|----------------|
| Register participant teams | Registration module captures required fields | Fully Achieved |
| Generate unique QR passes  | Unique ticket ID and QR/Base64 generation    | Fully Achieved |
| Persist records locally    | SQLAlchemy + SQLite implementation           | Fully Achieved |
| Protect gate and telemetry | Master passcode checks                       | Fully Achieved |
| Block duplicate check-ins  | Attendance flag and timestamp logic          | Fully Achieved |

## 4.7 Strengths & Limitations

### Strengths:

- Integrates registration, QR generation, verification and telemetry in one Python application.
- Uses unique ticket IDs and duplicate leader-email validation.
- Uses SQLite persistence without a third-party database dependency.
- Provides passcode-controlled gate verification and live attendance statistics.

### Limitations:

- Camera optical scanning requires HTTPS and explicit browser camera permissions on mobile devices.
- The master admin passcode is shared among organizers rather than using individual role accounts.
- Registration data is stored in a local SQLite file and requires active persistence handling for cloud deployments.
- Free cloud instances may experience cold-start delays when inactive for extended periods.

## 4.8 Project Planning, Roles & Budget

| Milestone                 | Activity                                    | Output                            |
|---------------------------|---------------------------------------------|-----------------------------------|
| 1                         | Requirement analysis and problem definition | Project scope and requirements    |
| 2                         | UI and registration workflow design         | Registration module               |
| 3                         | QR generation and database integration      | Ticket/pass storage               |
| 4                         | Gate verification and telemetry             | Check-in and admin modules        |
| 5                         | Testing and documentation                   | Final results and capstone report |
| Student                   | Assigned Responsibility                     | Actual Contribution               |
| Harini K                  | Registration and input validation           | 25%                               |
| Nithyasree P              | QR generation and pass handling             | 25%                               |
| S R Anusika               | Gate verification and attendance logic      | 25%                               |
| Keerthi P                 | Admin roster and telemetry                  | 25%                               |
| Item                      | Estimated Cost                              | Actual Cost                       |
| Python/software resources | No additional project cost                  | No additional project cost        |
| SQLite/local database     | No additional project cost                  | No additional project cost        |
| Cloud deployment          | Not specified in PPT                        | Not specified in PPT              |

# CHAPTER 5

## CONCLUSION, FUTURE WORK AND REFLECTION

### 5.1 Conclusion

The capstone delivered a functional lightweight event registration and QR check-in platform using Python. The system addresses manual registration and gate congestion by combining interactive team registration, unique digital QR passes, SQLite persistence, controlled optical verification and live attendance telemetry. The reported testing demonstrated correct QR payload matching, successful verification and duplicate-entry blocking. The project therefore provides a practical foundation for automated event gatekeeping while keeping the current implementation relatively simple and lightweight.

### 5.2 Future Work

- Replace the shared master passcode with salted-hash authentication and individual organizer accounts.
- Add automated email dispatch of QR passes through SMTP after successful registration.
- Migrate the database to PostgreSQL for distributed multi-gate check-in terminals.
- Integrate NFC/RFID smart-wristband pairing during initial gate QR verification.
- Extend testing for larger participant volumes and deployment environments.

### 5.3 Professional Learning & Individual Reflection

#### New Knowledge Acquired:

- Python web application development using FastHTML and structured input handling.
- QR-code generation, BytesIO buffering and Base64 image representation.
- SQLAlchemy ORM, SQLite persistence and attendance-state management.

#### Engineering & Professional Skills Developed:

- Modular program design and integration of multiple software components.
- Input validation, duplicate detection, authentication and database verification.
- Testing, interpretation of outputs and technical documentation.

#### Individual Reflection:

Harini K: I contributed to the team registration workflow and input validation. The project helped me understand how user-entered data must be checked before it is stored and used by later modules. A key learning was connecting the registration interface to persistent records. If repeated, I would improve validation and user feedback further.

Nithyasree P: I contributed to QR generation and pass handling. Working with unique ticket IDs, in-memory image buffers and Base64 encoding showed how data can move efficiently from Python logic to a web interface. The QR round-trip testing helped me understand verification of generated outputs. I would further automate test cases if the project were repeated.

S R Anusika: I contributed to gate verification and attendance logic. The main learning was how a scanned ticket can be checked against a database and how an attendance flag prevents duplicate entry. The passcode-controlled workflow also highlighted the importance of access control. I

would improve role-based authentication in a future version.

Keerthi P: I contributed to the admin roster and attendance telemetry. I learned how database records can be aggregated into useful event-level statistics such as registered teams and checked-in participants. The project improved my understanding of modular Python application design. I would extend the telemetry to support larger distributed deployments.

## REFERENCES

- [1] Patel, R., et al. (2021). "Design and Implementation of QR-Code Based Access Control Systems," IEEE Conference Publication, IEEE Xplore.
- [2] Zhang, L., et al. (2022). "A Comparative Study of Modern Web Frameworks and In-Memory Data Streaming," IEEE Conference Publication, IEEE Xplore.
- [3] Rao, K., et al. (2023). "Database Modeling and Schema Management Using Python ORMs," IEEE Conference Publication, IEEE Xplore.
- [4] Sharma, S., & Kumar, A. (2023). "A Survey on Automated Event Ticketing and Authentication Techniques," IEEE Conference Publication, IEEE Xplore.
- [5] Gupta, M., & Verma, P. (2024). "GUI and Web Application Development Using Modern Python Architectures," IEEE Conference Publication, IEEE Xplore.

The above references are the sources explicitly listed in the submitted capstone presentation.

# APPENDICES

## **Appendix A – Detailed Calculations**

Not applicable. The project is primarily a software implementation and does not contain mathematical engineering calculations.

## **Appendix B – Engineering Drawings / Schematics**

Conceptual system flow: Registration → Ticket ID → QR Generation → SQLite Storage → Admin Passcode → Camera Scan → Ticket Validation → Check-in / Duplicate Warning → Telemetry.

## **Appendix C – Source Code / Code Repository Information**

Key source-code elements presented in the PPT include `generate_qr_b64(ticket_id)`, `register_team(...)`, `checkin_user(ticket_id)`, `get_stats()` and `admin_logout()`. The complete project source code is maintained separately from this report.

## **Appendix D – Datasheets**

Not applicable. No hardware datasheets were presented.

## **Appendix E – Experimental / Raw Data**

The PPT reports software test outputs including QR round-trip matching, unauthorized-access blocking and sample live telemetry values.

## **Appendix F – Ethics / Institutional Approval**

The project is a software capstone focused on event registration and access verification. No human-subject experiment or sensitive research dataset is reported in the PPT.

## **Appendix G – Project Meeting / Progress Records**

Not included in the submitted PPT.

## **Appendix H – User/Industry Feedback**

Not included in the submitted PPT.

## **Appendix I – Publications / Patents / Competitions / Product Outcomes**

Not applicable based on the submitted PPT.

## **Appendix J – Individual Contribution Evidence**

The team contribution table records equal 25% contributions for Harini K, Nithyasree P, S R Anusika and Keerthi P. This equal split is an administrative assumption because the PPT does not specify individual percentages; it can be edited if the team has an official contribution split.

## CAPSTONE PROJECT OUTCOME MAPPING SHEET

| Outcome                                          | SO / Area   | Location in This Report              |
|--------------------------------------------------|-------------|--------------------------------------|
| Complex engineering problem formulation          | SO1         | Ch. 1, Ch. 2                         |
| Engineering design under constraints             | SO2         | Ch. 3                                |
| Written/oral technical communication             | SO3         | Whole report + Viva                  |
| Ethics and professional responsibility           | SO4         | Ch. 3 (3.7)                          |
| Teamwork and leadership                          | SO5         | Ch. 4 (4.8)                          |
| Experimentation, analysis, engineering judgment  | SO6         | Ch. 4 (4.3–4.6)                      |
| Independent acquisition of new knowledge         | SO7         | Ch. 5 (5.3)                          |
| Standards                                        | SO2         | Ch. 3 (3.2)                          |
| Sustainability and societal/environmental impact | SO2/SO4     | Ch. 3 (3.7)                          |
| Risk assessment and mitigation                   | SO2/SO4     | Ch. 3 (3.7)                          |
| Security considerations                          | Relevant SO | Ch. 3 (3.7)                          |
| Integrated/system-level engineering              | SO1/SO2     | Ch. 3 (3.5)                          |
| Project planning and management                  | SO5         | Ch. 4 (4.8)                          |
| Individual contribution                          | SO1–SO7     | Contribution Statement + Ch. 4 (4.8) |